Short Answer

	Kilde	Sluk	Max flyt
graf1	0	7	10
graf2	0	1	27
graf3	0	1	42
graf4	0	7	11
graf5	0	7	90

Maks flyt i *flytgraf3* er **42**, og i *flytgraf5* er det **90**.

FlytGraf5, Kilde 0 og Sluk 1:

```
Path 1: 0 -> 8 -> 9 -> 1  | added flow: 10  | total: 10
Path 2: 0 -> 2 -> 3 -> 9 -> 8 -> 14 -> 15 -> 1  | added flow: 10  | total: 20
Path 3: 0 -> 2 -> 3 -> 9 -> 8 -> 14 -> 15 -> 1  | added flow: 5  | total: 25
Path 4: 0 -> 2 -> 3 -> 4 -> 5 -> 8 -> 9 -> 12 -> 13 -> 14 -> 15 -> 1  | added flow:
10  | total: 35
Path 5: 0 -> 2 -> 3 -> 4 -> 5 -> 8 -> 9 -> 12 -> 13 -> 14 -> 15 -> 1  | added flow:
1  | total: 36
Path 6: 0 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 9 -> 8 -> 10 -> 11 -> 12 -> 13 -> 14 ->
15 -> 1  | added flow: 6  | total: 42
42
```

FlytGraf5, Kilde 0 og Sluk 7:

```
Path 1: 0 -> 2 -> 3 -> 7  | added flow: 10  | total: 10
Path 2: 0 -> 2 -> 4 -> 7  | added flow: 25  | total: 35
Path 3: 0 -> 5 -> 6 -> 7  | added flow: 10  | total: 45
Path 4: 0 -> 5 -> 6 -> 1 -> 7  | added flow: 40  | total: 85
Path 5: 0 -> 2 -> 4 -> 6 -> 1 -> 7  | added flow: 5  | total: 90
90
```

Longer Answer (Details and explantion of the code)

> Implementation of the code uses classes which contains references to other nodes and edges. It does not use a neighbor table and other similar data structures. It uses getter-methods and use of different lists to achieve certain principles (FIFO) and for traversing (BFS). It also uses certain API for file reading.

Implementation is similar to previous assignment, if you're the same person evaluating this.

The "objects" necessary are nodes, vertices (will be referred to as edges) and the actual Flowgraph containing nodes and edges. I will be going through each class individual to explain their function and purpose;

Node.java

The simplest of the objects are nodes, as they only contain an id for reference and any amount of edges. An class with id and a mutable list, with getter method represent this fine:

```
public class Node {
    private List<Edge> edges;
    private final int id;

    public Node(int id){
        this.edges = new ArrayList<>();
        this.id = id;
    }

    public int getId() {
        return id;
    }

    public List<Edge> getEdges() {
        return edges;
    }
}
```

Edge.java

Edges in a flowgraph are one-way pointer the have a an allowed flow and a capacity. Due to the nature of Edmond Karps algorithm, a reverse of that pointer might be necessary to use. Also, due to the nature of node contruction, edges must be mutable for make the flowgraph generation smoother. For example, when making an edge, we

need to make the reverse-edge afterwards and store eachothers referances respectivly. The edge class should have a method to calculate new flow value for Edmond Karps algorithm.

The contructor for Edge.java will not set a referance for the reverse edge upon creation; And should be set after creation during runtime:

```
public class Edge {
    private final int capacity;
    private int flow;
    private final Node from;
    private final Node to;
    private Edge reverse;

    public Edge(int capacity, Node from, Node to) {
        this.capacity = capacity;
        this.flow = 0;
        this.from = from;
        this.to = to;
    }

    public void setReverse(Edge reverse) {
        this.reverse = reverse;
    }

    // ...
}
```

These helper functions apply changes set by EdmondsKarp; for calculating rest and updating current flow:

```
public int rest(){return capacity - flow;}

public void setNewFlowStats(int ammount){
    flow += ammount;
    reverse.flow -= ammount;
}
```

Assume that every variable have their corresponding getter-methods.

FlowGraph.java

This class is container for all nodes and edges aswell as the node for source and sink, handles certain aspects of creating the graph and the EdmondKarp algorithm for getting maximum flow.

All nodes will be in a unsorted list but source and sink nodes will have their own referances stored as a variable;

```
public class FlowGraph {
    private List<Node> nodes;
    private Node source;
    private Node sink;

    public FlowGraph(List<Node> nodes) {
        this.nodes = nodes;
        this.source = nodes.get(0);
        this.sink = nodes.get(nodes.size() - 1);

    }

    public FlowGraph() {
        this.nodes = new ArrayList<>();
        this.source = null;
        this.sink = null;
    }
    // ...
}
```

This class also handles node creation, this is so that no duplicates of nodes occur; Due to the nature of the assignment, nodes follow an incremental pattern, making it possible iteratively create below its id until current id is created. This makes it so we can combine and getter function and a make function:

```
public Node getOrCreateNode(int id) {
    while (nodes.size() <= id) {
        addNode();
    }
    return nodes.get(id);
}

public Node addNode() {
    Node n = new Node(nodes.size());
    nodes.add(n);
    return n;
}
```

After creating the nodes, we need to assign a source and sink per assignment or corretly:

```
public void setSink(Node sink) {
    this.sink = sink;
}
```

```
public void setSource(Node source) {
    this.source = source;
}
```

@toString method overrriden to get make it possible to print out the edges, (and current flow of them, but mostly used to check if edges are made corretly):

```
@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    for (Node n : nodes){
        for (Edge e : n.getEdges()){
            if (e.getCapacity() > 0) {   // skip the reverse edges
                sb.append(n.getId()).append(" -> ").append(e.getTo().getId())
                        .append("
("").append(e.getFlow()).append("/").append(e.getCapacity()).append(")\n");
            }
        }
    }
    return sb.toString();
}
```

Edmond Karps Algorithm

Step 1: BFS for shortest path To achieve a BFS we need a queue that follows the principle of FIFO, First in first out. This is done by using the `ArrayDeque` for its `add(x)` and `poll()` methods to add in the back and pull the first. The path is storred in a array containing edges. If statement checks if the next node for each edge is not the source, does have a rest bigger than 0, and is not an edge it has been before:

```
// BFS: find shortest path, remembering the edge used to reach each node
Edge[] parentEdge = new Edge[nodes.size()];
Deque<Node> queue = new ArrayDeque<>();
queue.add(source);

while (!queue.isEmpty() && parentEdge[sink.getId()] == null) {
    Node cur = queue.poll();
    for (Edge e : cur.getEdges()) {
        Node next = e.getTo();
        if (e.rest() > 0 && next != source && parentEdge[next.getId()] == null) {
            parentEdge[next.getId()] = e;
            queue.add(next);
        }
    }
}
```

```
if (parentEdge[sink.getId()] == null) break;    // no augmenting path left
```

Step 2: Going backwards to find the lowest capasity/flow (the bottleneck): By using the parentEdge array, is goes backwards from sink to source comparing every value. The ramining is the bottleneck:

```
// Find bottleneck by going backwards from sink
int bottleneck = Integer.MAX_VALUE;
Deque<Node> path = new ArrayDeque<>();
for (Node n = sink; n != source; n = parentEdge[n.getId()].getFrom()) {
    bottleneck = Math.min(bottleneck, parentEdge[n.getId()].rest());
    path.addFirst(n);
}
path.addFirst(source);
```

Step 3: Set new stats Go through each edge and push new changes using helper methods in edges:

```
// Calculate new flow
for (Node n = sink; n != source; n = parentEdge[n.getId()].getFrom()) {
    parentEdge[n.getId()].setNewFlowStats(bottleneck);
}
```

During each iterasjon the bottleneck is added to a variable called maxFlow. The resulting value is the Max Flow for the Flow Graph!